CS 180 · PROJECT 4 · PART A

Image Warping & *Mosaicing*

Shooting photos, recovering homographies, warping and stitching them into panoramas.

A.1 Shoot | A.2 Homographies | A.3 Warp | A.4 Mosaic

A.1 Shoot the Pictures

SAME SPOT, JUST ROTATE THE CAMERA

For each set of photos, I found a spot, stayed there, and rotated my phone, making sure there was enough overlap to find the correspondence points. I scaled things down to cut the run time, since the phone stores photos at high resolution. I also took a couple of photos with obvious rectangles (not part of this deliverable, but I thought I would put them here).

SET 1 · BUILDING FACADE



1A left



2A right

SET 2 · ROTUNDA



3A left



4A right

SET 3 · ATRIUM



5A left



6A right

RECTIFICATION PHOTOS



7A gold mirror frame



8A exit stair sign on the door

A.2 Recover Homographies

P' = HP, 8 UNKNOWNS

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -ux & -uy \\ 0 & 0 & 0 & x & y & 1 & -vx & -vy \end{bmatrix} \begin{bmatrix} h_1 \\ \vdots \\ h_8 \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix}, \qquad h_9 = 1$$

We know from lecture that the homography has 8 degrees of freedom (the last entry is fixed to be 1). Following the equation, we can isolate everything in terms of u and v. Then we can set up a system of linear equations (choosing 9 to 10 correspondences to make sure it is overdetermined) and solve it with least squares to get the homographies.

SET 1 · BUILDING FACADE · 10 CORRESPONDENCES



1B clicked correspondences, left: image 1 (warped) · right: image 2 (reference)

SET 2 · ROTUNDA · 9 CORRESPONDENCES



2B clicked correspondences, left: image 1 (warped) · right: image 2 (reference)

SET 3 · ATRIUM · 9 CORRESPONDENCES



3B clicked correspondences, left: image 1 (warped) · right: image 2 (reference)

THE SYSTEM FOR SET 1

Each of the 10 correspondences gives two rows, so A is 20 × 8, solved with `np.linalg.lstsq`.

$$
\begin{bmatrix}
849 & 135 & 1 & 0 & 0 & 0 & -431292 & -68580 \\
0 & 0 & 0 & 849 & 135 & 1 & -151122 & -24030 \\
858 & 171 & 1 & 0 & 0 & 0 & -440154 & -87723 \\
0 & 0 & 0 & 858 & 171 & 1 & -181038 & -36081 \\
474 & 102 & 1 & 0 & 0 & 0 & -60672 & -13056 \\
0 & 0 & 0 & 474 & 102 & 1 & -38394 & -8262 \\
474 & 118 & 1 & 0 & 0 & 0 & -60672 & -15104 \\
0 & 0 & 0 & 474 & 118 & 1 & -46926 & -11682 \\
915 & 497 & 1 & 0 & 0 & 0 & -507825 & -275835 \\
0 & 0 & 0 & 915 & 497 & 1 & -461160 & -250488 \\
947 & 496 & 1 & 0 & 0 & 0 & -548313 & -287184 \\
0 & 0 & 0 & 947 & 496 & 1 & -473500 & -248000 \\
780 & 275 & 1 & 0 & 0 & 0 & -347100 & -122375 \\
0 & 0 & 0 & 780 & 275 & 1 & -232440 & -81950 \\
805 & 520 & 1 & 0 & 0 & 0 & -375130 & -242320 \\
0 & 0 & 0 & 805 & 520 & 1 & -424235 & -274040 \\
479 & 557 & 1 & 0 & 0 & 0 & -57001 & -66283 \\
0 & 0 & 0 & 479 & 557 & 1 & -283089 & -329187 \\
534 & 173 & 1 & 0 & 0 & 0 & -106266 & -34427 \\
0 & 0 & 0 & 534 & 173 & 1 & -90246 & -29237
\end{bmatrix}
\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{bmatrix}
=
\begin{bmatrix} 508 \\ 178 \\ 513 \\ 211 \\ 128 \\ 81 \\ 128 \\ 99 \\ 555 \\ 504 \\ 579 \\ 500 \\ 445 \\ 298 \\ 466 \\ 527 \\ 119 \\ 591 \\ 199 \\ 169 \end{bmatrix}
$$

RECOVERED HOMOGRAPHIES

SET 1 · BUILDING FACADE

$$
H = \begin{bmatrix} 1.895 & -0.05696 & -709.9 \\ 0.3911 & 1.553 & -228.6 \\ 9.059 \times 10^{-4} & -7.043 \times 10^{-5} & 1 \end{bmatrix}
$$

SET 2 · ROTUNDA

$$
H = \begin{bmatrix} 0.553 & 0.00557 & 333.6 \\ -0.2185 & 0.8167 & 105.2 \\ -4.191 \times 10^{-4} & -5.610 \times 10^{-5} & 1 \end{bmatrix}
$$

SET 3 · ATRIUM

$$
H = \begin{bmatrix} 1.927 & -0.07878 & -728.2 \\ 0.4009 & 1.483 & -175.9 \\ 9.890 \times 10^{-4} & -2.343 \times 10^{-4} & 1 \end{bmatrix}
$$

A.3 Warp the Images

INVERSE WARP, NO HOLES

$$
\begin{bmatrix} x \\ y \\ w \end{bmatrix} = H^{-1} \begin{bmatrix} x_{out} \\ y_{out} \\ 1 \end{bmatrix}, \qquad \text{bilinear: } (1-d_x)(1-d_y)I_{00} + d_x(1-d_y)I_{01} + (1-d_x)d_yI_{10} + d_xd_yI_{11}
$$

We use inverse warping to avoid holes. I first find the four corners and then make a bounding box around those corners. Then I go through every pixel, apply the inverse homography, and look at where it came from. There are two different ways to fill it in: nearest neighbor or bilinear interpolation (anything outside becomes black). For rectification, I clicked the four corners, made a guess of the target rectangle, and then warped it.

RECTIFICATION 1 · GOLD MIRROR FRAME

The four inner corners of the frame are sent to a 125 × 150 rectangle.



1C chosen corners



2C nearest neighbor



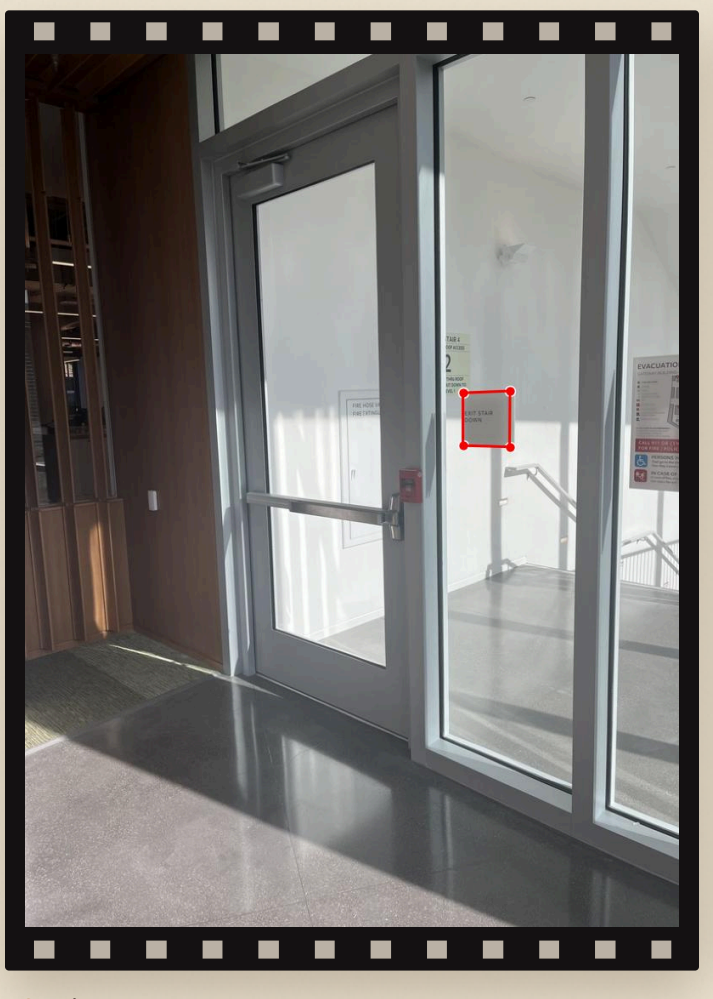
3C bilinear

RECT 1

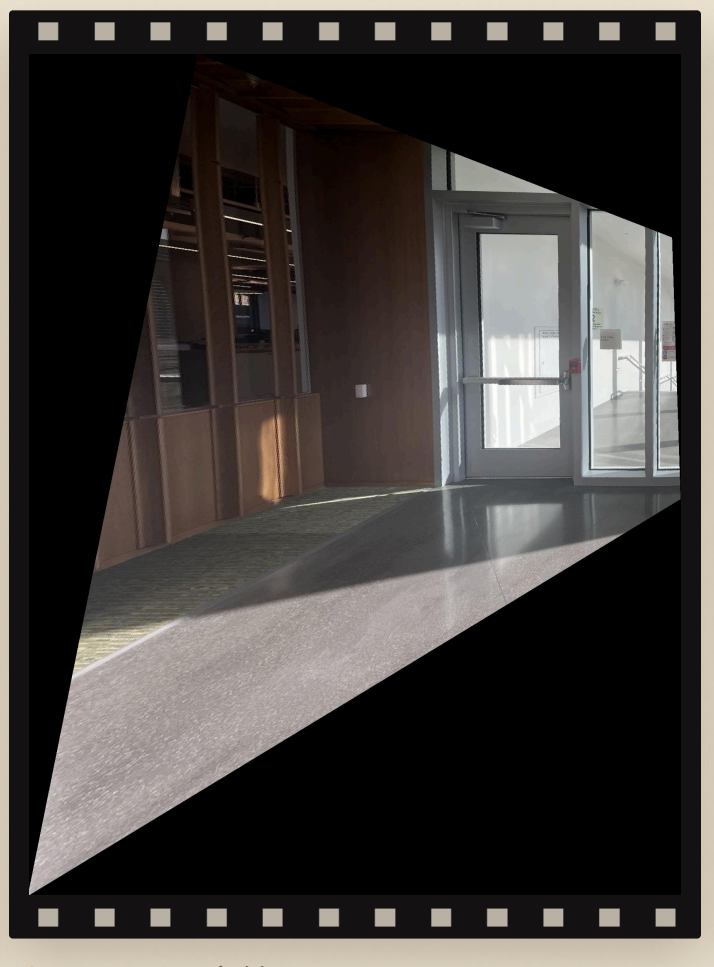
$$H = \begin{bmatrix} 0.725 & 0.1808 & -326.6 \\ -0.2182 & 0.6617 & 75.92 \\ 7.575 \times 10^{-5} & -3.798 \times 10^{-4} & 1 \end{bmatrix}$$

RECTIFICATION 2 · EXIT STAIR SIGN

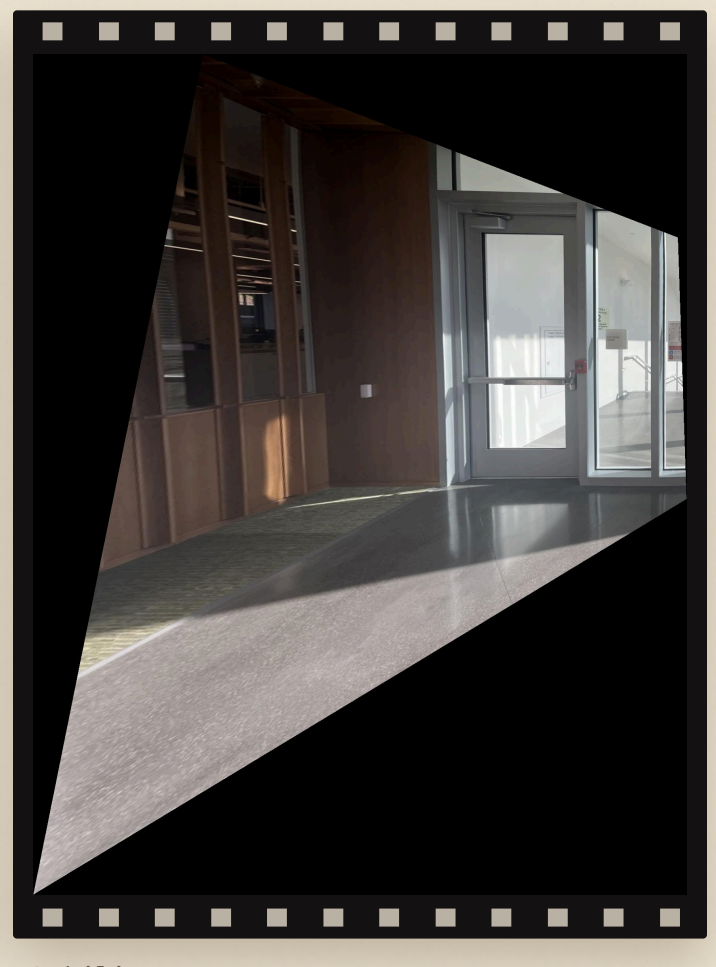
The four corners of the sign are sent to a 50 × 50 square.



4C chosen corners



5C nearest neighbor



6C bilinear

RECT 2

$$H = \begin{bmatrix} 2.014 & 0 & -1.019 \times 10^{3} \\ 0.06965 & 1.687 & -694.8 \\ 0.002338 & -2.942 \times 10^{-4} & 1 \end{bmatrix}$$

NEAREST NEIGHBOR VS. BILINEAR, UP CLOSE

rect 1 **1012 × 942** · NN **3.0 s** · bilinear **4.7 s**

rect 2 **1653 × 2123** · NN **10.4 s** · bilinear **15.8 s**



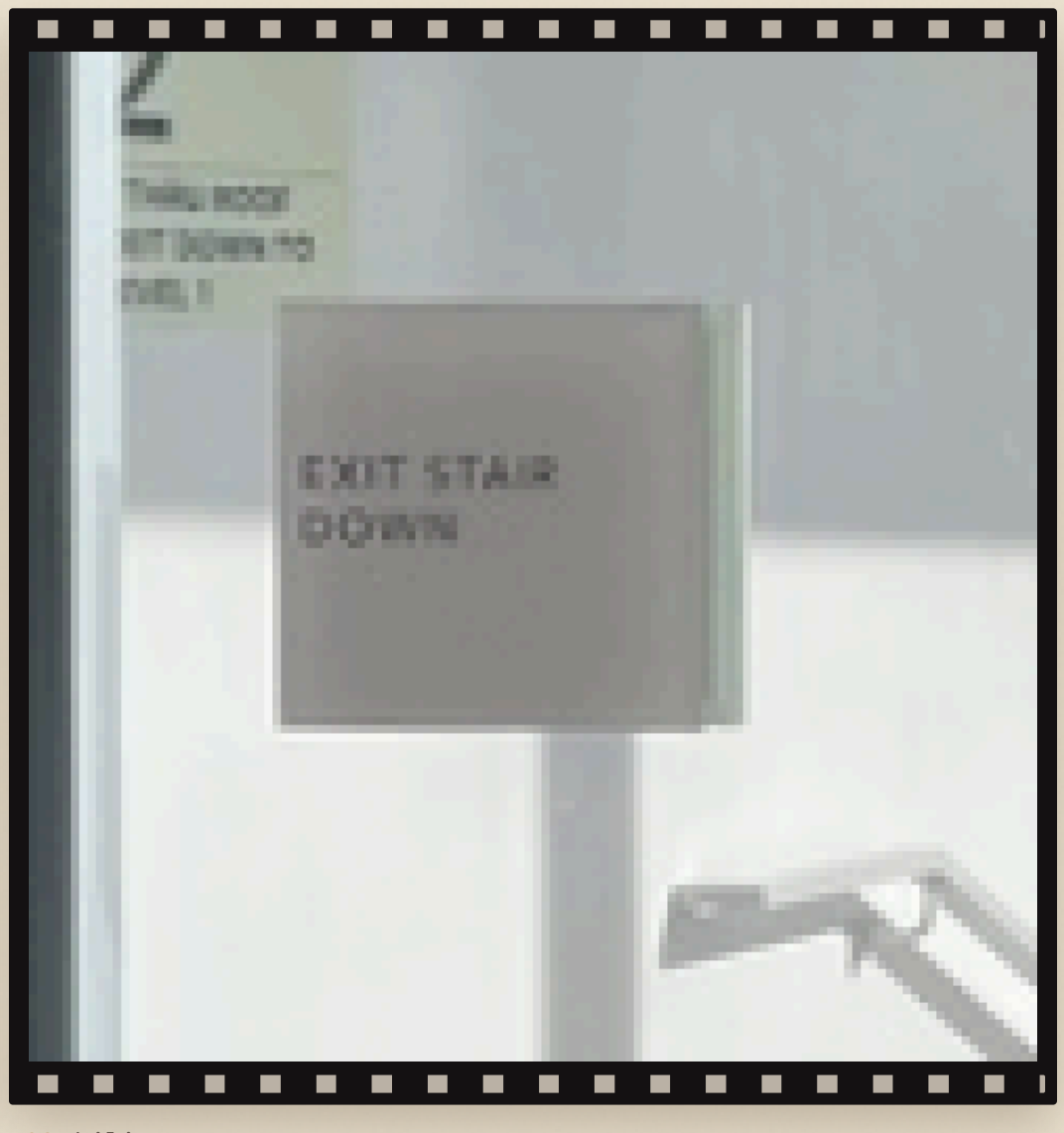
7C nearest neighbor · 5× crop



8C bilinear · 5× crop



9C nearest neighbor · 5× crop



10C bilinear · 5× crop

I noticed nearest neighbor, because it just takes the one closest pixel, tends to be more jagged, while bilinear interpolation looks at a couple of pixels and tries to create a smoother result. Speed-wise, bilinear was about 1.5 to 1.7× slower since it reads four pixels instead of one and does a bit more math per pixel. That is a small price for how much better it looks, so I would use bilinear unless speed really mattered, and that is what I used for the mosaics.

A.4 Blend into a Mosaic

FEATHER THE SEAMS

$$\alpha_i = \frac{\mathrm{dist}_i}{\mathrm{dist}_1 + \mathrm{dist}_2}, \qquad \mathrm{out} = \alpha_1 L_1 + \alpha_2 L_2 + \begin{cases} D_1 & \mathrm{dist}_1 > \mathrm{dist}_2 \\ D_2 & \text{otherwise} \end{cases}, \qquad L = G_\sigma * I, \;\; D = I - L$$

I use the second photo as the reference, then warp the first one with H using bilinear. The canvas is resized to fit both, using the warped corners. If you just paste one on top of the other, you get a hard seam. Instead, we do feather weighting: pixels close to the middle of an image get a lot of weight, and it fades to 0 toward the outside (each pixel's weight is its distance to the edge of its own image, divided by the total from both images, so in the overlap the warped image slowly hands off to the reference as you move toward its border). I also do a 2-level Laplacian blend (Gaussian with σ = 4): the high frequencies stay from whichever image is more inside at that pixel, while we blend the lower frequencies with the weights.

MOSAIC 1 · BUILDING FACADE



4D source A



5D source B



6D blended mosaic

This one came out clean. The columns and the roofline stay straight across the overlap, and the tree, which I was worried about since leaves are hard to line up, does not really ghost. The black wedges at the top and bottom are just where the warped photo tilts out, which is normal for a planar mosaic.

MOSAIC 2 · ROTUNDA

7D source A

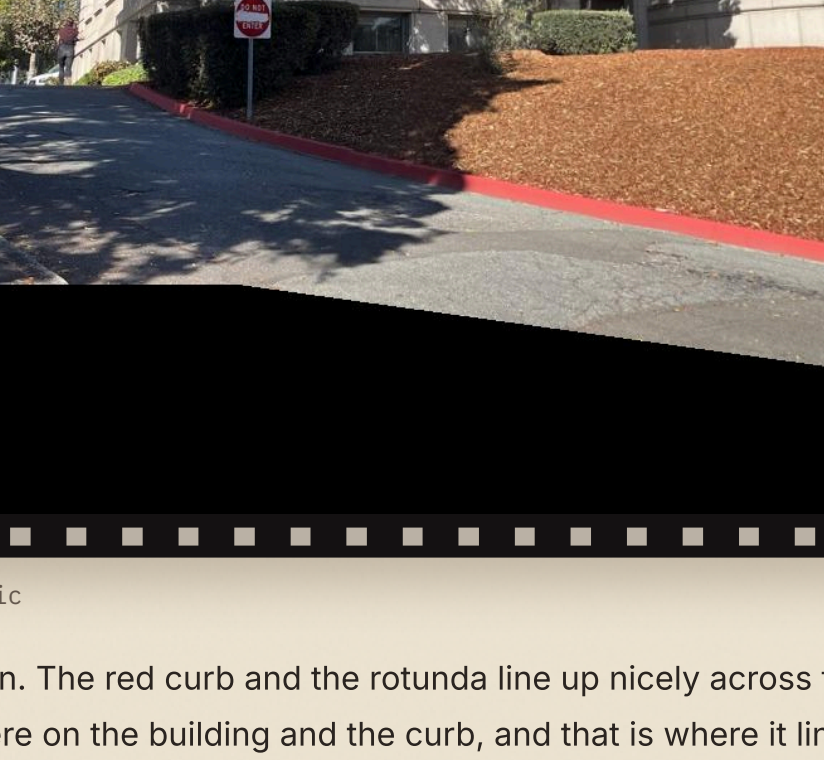
8D source B



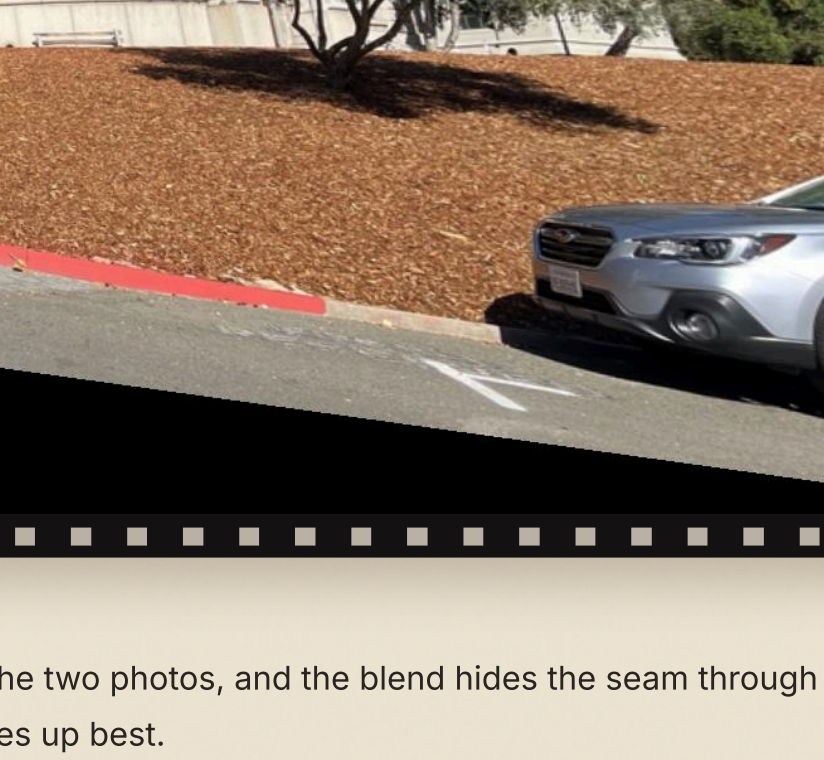
9D blended mosaic

Also pretty clean. The red curb and the rotunda line up nicely across the two photos, and the blend hides the seam through the trees. Most of my points were on the building and the curb, and that is where it lines up best.

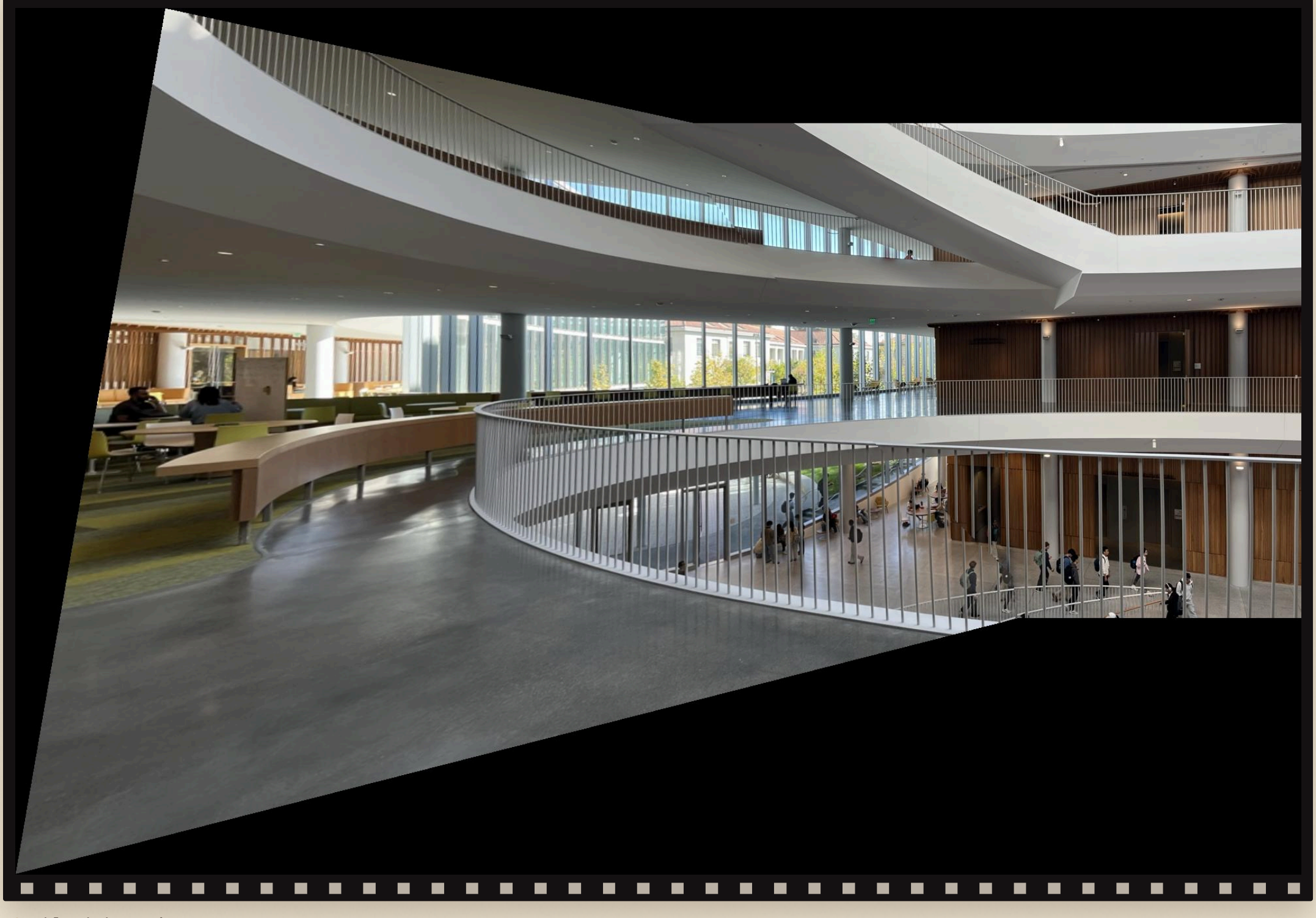
MOSAIC 3 · ATRIUM



10D source A



11D source B



12D blended mosaic

The curved railing actually lines up across both photos, which I was a little nervous about since curves make any misalignment really obvious. The people walking around on the floor below moved between shots, so there is a bit of a blur on them due to the motion.

END OF ROLL